PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Учебный проект использует синтетические обращения и не содержит персональных данных.

При выполнении работы следует использовать только приведённые или самостоятельно составленные учебные обращения. Реальные имена, контакты, номера учётных записей и другие сведения о пользователях для сценарного тестирования не требуются.

1. Паспорт самостоятельной работы

Содержание

- 2. Семь проверяемых шагов
- 3. Рекомендуемое распределение времени
- 4. Что такое пользовательский сценарий
- 5. Архитектура проекта CampusSupport Assistant
- 6. Требования к поведению решения
- 7. Исходная версия функции
- 9. Что нужно отправить преподавателю
- 10. Чек-лист перед сдачей
- 11. Итог работы

Параметр	Описание
Тема	4.12. Тестирование по сценарию пользователя
Вид деятельности	Самостоятельная работа без непосредственного участия преподавателя
Продолжительность	90 минут
Цель	Проверить прикладное решение целостными пользовательскими сценариями: от цели пользователя до полного результата.
Итог	Семь самостоятельных результатов: архитектура, прогноз, сценарии, протокол, исправление, разбор ошибок и улучшения.

Параметр	Описание
Инструменты	Python 3 или ручная трассировка; документ или тетрадь; интернет не требуется.

Работа проверяет не только умение запустить функцию, но и способность заранее определить правильное поведение системы. Поэтому ожидаемый результат, фактический результат и объяснение расхождения оформляются как разные части анализа.

> **Важно для сдачи**
>
> Проект CampusSupport Assistant является единым контекстом, но работа состоит из семи самостоятельных шагов. Каждый шаг оформляется отдельно и оценивается независимо. Если один из шагов опирается на результат предыдущего, его всё равно необходимо представить как отдельный завершённый результат. Например, исправленная функция не заменяет протокол тестирования, а протокол не заменяет описание требований.

2. Семь проверяемых шагов

№	Шаг	Что нужно сдать
1	Архитектура и требования	Входы, выход, допустимые значения, порядок правил и критерий успеха сценария.
2	Прогноз исходной версии	Ожидаемый и фактический результат для четырёх сценариев с объяснением.
3	Собственные сценарии	Четыре воспроизводимых сценария: обычный, критический, неоднозначный/низкая уверенность и некорректный.
4	Сценарный протокол и метрики	Результаты восьми обязательных сценариев и три показателя качества.
5	Исправление и автотесты	Исправленная функция process_request или полный псевдокод.
6	Разбор ошибок	Три карточки: симптом, нарушенное требование, первопричина, последствие, исправление и ретест.

№	Шаг	Что нужно сдать
7	Улучшения и готовность	Два улучшения, риски и вывод о готовности к учебной демонстрации.

Последовательность шагов отражает полный цикл проверки: сначала фиксируется контракт, затем обнаруживаются расхождения, после этого вносятся изменения и подтверждается их результат. Пропуск ожиданий или ретеста делает вывод об исправности решения недоказанным.

3. Рекомендуемое распределение времени

Этап	Время
Ознакомление с проектом	10 минут
Шаг 1	10 минут
Шаг 2	10 минут
Шаг 3	10 минут
Шаг 4	10 минут
Шаг 5	10 минут
Шаг 6	10 минут
Шаг 7	10 минут
Самопроверка и сдача	10 минут

Распределение является ориентиром. Если ручная трассировка занимает больше времени, полезно сначала заполнить ожидаемые результаты и причины, а затем вернуться к фактическому выполнению. Финальные 10 минут следует сохранить для проверки согласованности таблиц, функции и вывода.

4. Что такое пользовательский сценарий

Пользовательский сценарий включает роль, цель, контекст, входные данные, ожидаемый полный результат и критерий успеха. Сценарий считается успешным только при совпадении всех трёх полей результата: category, priority и action.

Один тестовый вход отвечает на вопрос «что передано функции», а сценарий дополнительно показывает, зачем пользователь обратился и какое завершённое поведение системы считается правильным. Например, правильная категория при неверном приоритете не решает сценарий полностью: обращение будет распознано, но может попасть в неподходящую очередь обработки.
Критерий успеха является строгим: частичное совпадение не засчитывается как успешный сценарий. При этом отдельные совпавшие поля можно учитывать при разборе первопричины, но не при расчёте Scenario Pass Rate.

5. Архитектура проекта CampusSupport Assistant

Поле	Допустимые значения	Назначение
text	непустая строка после strip()	Текст обращения пользователя.
role	student, teacher, administrator	Роль пользователя.
channel	portal, email, phone	Канал обращения.
confidence	число от 0 до 1; bool недопустим	Уверенность модели в выбранной категории.

Контракт входа включает не только перечень полей, но и их типы, диапазоны и допустимые значения. Например, строка из пробелов формально является строкой, однако после strip() становится пустой и поэтому считается некорректным text.
Для confidence важны обе границы диапазона: значения 0 и 1 допустимы, а значения меньше 0 или больше 1 должны быть отклонены. Значения типа bool недопустимы отдельно, даже несмотря на то, что в Python bool связан с числовыми типами.
Функция возвращает словарь с полями category, priority и action.

Поле результата	Допустимые значения
category	access, network, equipment, software, other
priority	P1, P2, P3
action	auto_route, operator_review, input_error

Поля результата описывают разные стороны решения. category определяет тему обращения, priority — срочность обработки, а action — дальнейший способ обработки. Поэтому значение other не всегда означает ошибку входа: оно может появиться и при неоднозначном тексте, но тогда action должен показывать operator_review, а не input_error.

6. Требования к поведению решения

ID	Требование
R1	Если тип входа неверен, confidence вне 0–1, text пуст после strip(), role или channel неизвестны — вернуть other/P3/input_error.
R2	Перед анализом выполнить text.strip().lower().
R3	Категория access: пароль, вход, доступ, учётная запись.
R4	Категория network: сеть, wi-fi, интернет, сервер недоступен.
R5	Категория equipment: принтер, ноутбук, компьютер, клавиатура.
R6	Категория software: программа, приложение, обновление, ошибка установки.
R7	Если обнаружены две и более категории — category=other и action=operator_review.
R8	Если confidence < 0.60 — action=operator_review. При confidence=0.60 автоматическая обработка допустима.
R9	Фразы «сервер недоступен», «утечка данных», «не работает сеть во всём корпусе» задают priority=P1 независимо от роли и канала.
R10	В остальных случаях teacher, administrator или channel=phone получают P2; остальные — P3. При валидном вводе без R7/R8 action=auto_route.

Требования необходимо применять в согласованном порядке. Сначала проверяется валидность всех входов по R1. Только для валидного обращения выполняются нормализация, поиск категорий, определение действия и приоритета.

R7 и R8 изменяют action по разным причинам: R7 указывает на неоднозначность содержания, а R8 — на недостаточную уверенность. В обоих случаях обращение передаётся оператору, но низкая уверенность сама по себе не требует заменять найденную единственную категорию на other.

Приоритет определяется независимо от action. Поэтому критическая фраза должна дать P1 даже тогда, когда обращение дополнительно требует operator_review из-за нескольких категорий или низкой уверенности. Если критической фразы нет, применяется правило роли и канала из R10.

> **Обратите внимание на границы**
>
> Условие confidence < 0.60 не включает значение 0.60. Не следует заменять строгий знак < на <=, иначе граничный валидный сценарий будет ошибочно направлен оператору. Также нельзя останавливать поиск после первого ключевого слова: для выполнения R7 необходимо собрать все обнаруженные категории и проверить их количество.

7. Исходная версия функции

```
def process_request(text, role, channel, confidence):
text = text.lower()
if confidence <= 0.60:
return {"category": "other", "priority": "P3", "action": "operator_review"}

if "пароль" in text or "вход" in text:
category = "access"
elif "сеть" in text or "интернет" in text:
category = "network"
elif "принтер" in text or "компьютер" in text:
category = "equipment"
elif "программа" in text or "обновление" in text:
category = "software"
else:
category = "other"

if role == "administrator":
priority = "P1"
elif role == "teacher":
priority = "P2"
else:
priority = "P3"

return {"category": category, "priority": priority, "action": "auto_route"}
```

> **Правило выполнения**

> Сначала определяйте ожидаемый результат по требованиям R1–R10. Только затем выполняйте исходный код или ручную трассировку. Не заменяйте ожидание фактическим ответом ошибочной версии.
> При ручной трассировке записывайте, какие условия проверены, какая ветвь выбрана и на каком return завершилась функция. Если выполнение прерывается исключением до формирования словаря, фактический результат следует обозначить как «сбой программы».

Исходная версия намеренно не полностью соответствует контракту. Ошибки могут находиться не только в списках ключевых слов, но и в порядке операций, знаке сравнения, обработке нескольких совпадений, проверке входов и назначении приоритета.

9. Что нужно отправить преподавателю

- шаг 1 — архитектуру и требования;
- шаг 2 — таблицу прогнозов;
- шаг 3 — четыре собственных сценария;
- шаг 4 — протокол T01–T08 и три метрики;
- шаг 5 — исправленную функцию или полный псевдокод;
- шаг 6 — три карточки ошибок;
- шаг 7 — два улучшения и вывод о готовности;
- ответы на самопроверку или отметку о сверке с ключом.

Перед отправкой следует проверить, что идентификаторы сценариев и требований используются последовательно. Если в карточке ошибки указан ретест, соответствующий фактический результат должен быть доступен в протоколе или приведён в самой карточке.

10. Чек-лист перед сдачей

☐ Я оформил семь отдельных пронумерованных шагов.
☐ Я записал ожидания до запуска исходной функции.
☐ Мои четыре сценария проверяют разные цели и риски.
☐ В протоколе успех означает совпадение всех трёх полей.
☐ Я проверил типы, диапазон confidence и пустой text.
☐ Я учёл границу confidence=0.60 и несколько категорий.

☐ Я подтвердил исправления повторными тестами.
☐ Я отличил обязательные исправления от дальнейших улучшений.
☐ Я проверил, что метрики рассчитаны по правильным группам сценариев.
☐ Я убедился, что итоговый вывод подтверждается протоколом и ретестами.

11. Итог работы

Вы выполнили семь самостоятельных действий сценарного тестирования: описали контракт, спрогнозировали поведение, создали сценарии, рассчитали показатели, исправили функцию, разобрали первопричины и предложили улучшения. Такой формат позволяет проверить каждый обязательный результат отдельно.
Главный результат работы — не только исправленный код, но и прослеживаемая связь между пользовательской целью, требованием, тестом и выводом. Если эта связь сохранена во всех семи шагах, решение можно обоснованно оценивать по наблюдаемому поведению, а найденные ошибки — воспроизводить и проверять после исправления.